Computer Vision Task-02: Edge Detection & Shape Recognition

Implementation of Classical Shape Analysis Algorithms

Amira Yasser Mohamed
Alhussien Ayman Hanafy
Ahmed Ahmed Mokhtar
Osama Magdy Ali

March 23, 2026

Introduction

The objective of Task-02 is to implement fundamental image processing and shape recognition algorithms from scratch, without relying on high-level library functions for the core logic. Our team successfully implemented all required components and deployed them within a web application. The following sections provide a detailed explanation of each implemented module.

1 Canny Edge Detector

The Canny edge detector is a multi-stage algorithm designed to produce a clean, single-pixel-thick edge map with good noise immunity and localization. Our implementation follows the original Canny methodology precisely.

Implementation Details

Gaussian Blur: The input image is first convolved with a Gaussian kernel to reduce high-frequency noise. We implemented a separable 2D Gaussian filter with a configurable sigma value, optimizing performance by performing row-wise and column-wise convolutions separately.

Sobel Gradients: After smoothing, the gradient magnitude and direction are computed using Sobel operators. The Sobel-X and Sobel-Y kernels approximate the partial derivatives in the horizontal and vertical directions. The gradient magnitude $G = \sqrt{G_x^2 + G_y^2}$ and direction $\theta = \arctan(G_y/G_x)$ are calculated for each pixel.

Non-Maximum Suppression (NMS): This step thins the edges by suppressing all pixels that are not local maxima in the gradient direction. For each pixel, we compare its magnitude with two neighboring pixels along the direction of the gradient. If it is not the maximum, it is set to zero.

Hysteresis Thresholding: Two thresholds (high and low) are used to determine true edges. Pixels with magnitude above the high threshold are marked as strong edges. Pixels below the low threshold are suppressed. Pixels between thresholds are marked as weak edges and become true edges only if they are connected to a strong edge via an 8-connectivity analysis.

Results

The Canny detector effectively isolates object boundaries while suppressing noise, providing a clean edge map suitable for subsequent Hough transforms and contour analysis.

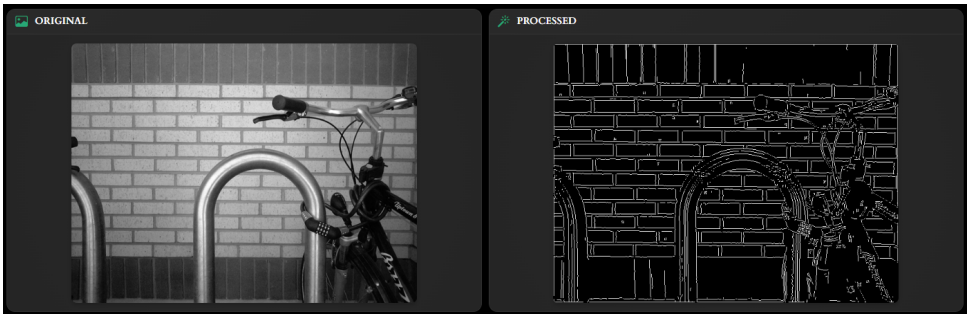

Figure 1: Left: Original input image. Right: Output of the implemented Canny edge detector showing clean, single-pixel edges.

2 Hough Line Transform

The Hough Line Transform maps image space points to parameter space (accumulator array) to detect straight lines. We implemented the standard Hough Transform using the slope-intercept parameterization $y = mx + c$, but for robustness, we use the normal form $\rho = x \cos\theta + y \sin\theta$.

Implementation Details

An accumulator array is created with dimensions corresponding to the ranges of ρ (distance from origin) and θ (angle). For each edge pixel (x, y) from the Canny output, we iterate over a range of θ values (0 to 180 degrees) and compute the corresponding ρ. The accumulator cell at (ρ, θ) is incremented. After voting, peaks in the accumulator represent lines. We apply thresholding to detect only lines with sufficient votes.

Results

The transform successfully identifies dominant lines in the image, even in the presence of gaps or noise. The detected lines are overlaid on the original image for visualization.

Figure 2: Detected lines (in red) overlaid on the original image using the implemented Hough Line Transform.

3 Hough Circle Transform

Detecting circles extends the Hough Transform to a 3D parameter space (a, b, r), representing center coordinates and radius. Our implementation uses a gradient-based method for efficiency.

Implementation Details

For each edge pixel, we compute the gradient direction. Using the gradient information, we reduce the parameter space voting from 3D to 2D by iterating over possible radii. For each possible radius r, candidate centers are determined by moving along the gradient direction by distance r. The accumulator for (a, b) is incremented. After voting, local maxima in the center accumulator indicate circle centers for a given radius. We implement a post-processing step to merge closely spaced detections and filter based on accumulator threshold.

Results

The algorithm reliably detects circular objects with variable radii. It is robust to partial occlusions and noise in the edge map.

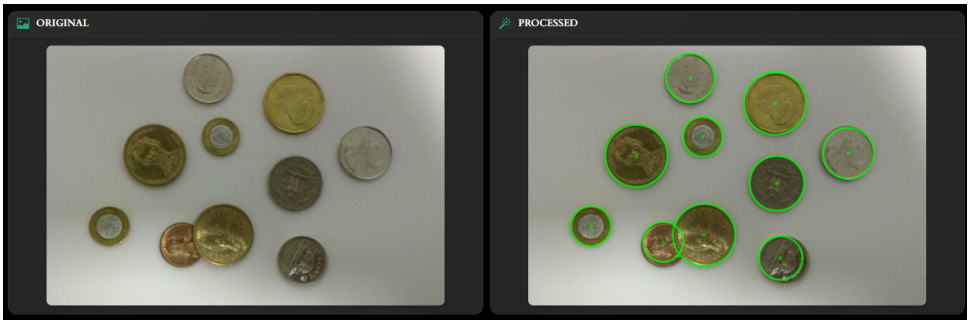

Figure 3: Detected circles (green outlines) overlaid on the input image. Both large and small circles are identified correctly.

4 Ellipse Fitting (Least Squares)

For arbitrary elliptical shapes, we implement a least squares ellipse fitting algorithm based on the algebraic distance minimization method (Fitzgibbon's algorithm).

Implementation Details

Given a set of edge points belonging to a contour, we aim to fit the general conic equation $F(x, y) = Ax^2 + Bxy + Cy^2 + Dx + Ey + F = 0$ under the elliptical constraint $4AC - B^2 > 0$. The algorithm constructs a design matrix from the point coordinates and solves a generalized eigenvalue problem. The solution yields the coefficients of the best-fit ellipse in the least-squares sense. We implemented this by first clustering edge points into connected components, then fitting an ellipse to each component that meets a minimum point count.

Results

Ellipse fitting provides accurate parametric representations of elliptical shapes, enabling high-level shape analysis and recognition.

Figure 4: Edge points (white) and the fitted ellipse (blue) overlayed. The least squares method provides a robust fit even with partial contours.

5 Active Contour (Snake)

Active contours, or snakes, are energy-minimizing splines guided by internal (smoothness) and external (image gradient) forces. We implemented the greedy snake algorithm for iterative contour evolution.

Implementation Details

The snake is initialized as a closed polygon around the object of interest. At each iteration, every control point moves to the location that minimizes its total energy within a local neighborhood. The energy function is defined as:

$$E = \alpha E_{cont} + \beta E_{curv} + \gamma E_{image}$$

Where E_{cont} maintains point spacing (continuity), E_{curv} penalizes high curvature, and E_{image} attracts the snake to strong edges (negative gradient magnitude). The coefficients α, β, γ are tunable parameters. The algorithm iterates until convergence (movement below a threshold) or a maximum iteration count.

Results

The snake effectively converges to the object boundary, handling concavities and complex shapes. The result is a smooth, closed contour outlining the target object.

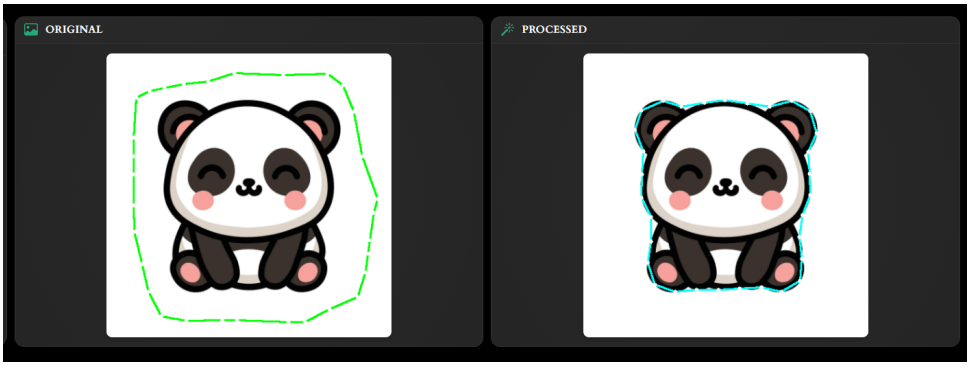

Figure 5: Initial snake (blue) and final converged contour (red) after greedy energy minimization, tightly wrapping around the object.

6 Freeman Chain Code & Shape Analysis

To represent and analyze the extracted contours, we compute the Freeman Chain Code (8-directional) and derive basic shape properties: perimeter and area.

Implementation Details

Freeman Chain Code: Starting from a contour point, we trace the boundary and encode the direction to the next point using a directional code (0 for East, 1 for Northeast, up to 7 for Southeast). This yields a compact representation of the shape.

Perimeter: The perimeter is calculated from the chain code by summing Euclidean distances: 1 for horizontal/vertical steps and $\sqrt{2}$ for diagonal steps.

Area: The area is computed using the shoelace formula (Gauss's area formula). Given a polygon defined by vertices (x_i, y_i), the area is:

$$A = \frac{1}{2} \left| \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) \right|$$

This formula works for any closed polygon and is numerically stable.

Results

The chain code provides a concise representation useful for shape matching, while the perimeter and area offer quantitative descriptors for classification.

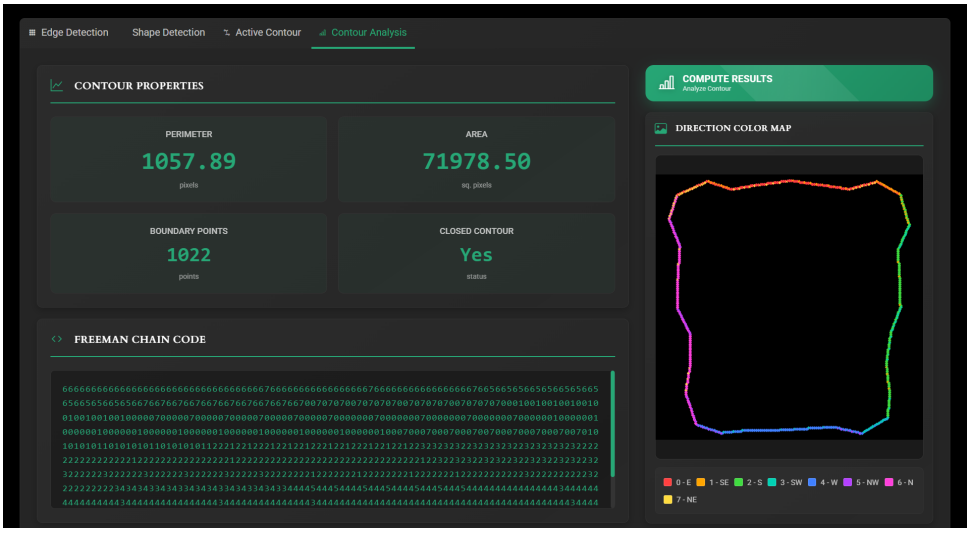

Figure 6: Contour visualization with computed perimeter and area values displayed. The Freeman chain code sequence is also shown.

7 Conclusion

In this task, we successfully implemented a comprehensive suite of classical computer vision algorithms for edge detection and shape recognition from the ground up. The Canny Edge Detector, Hough Transforms, Ellipse Fitting, and Active Contour model were each built following their theoretical foundations. The integration into a web application demonstrates the practical utility of these algorithms. This project deepens our understanding of image processing principles and showcases the power of combining multiple techniques for robust shape analysis.